

Lezione 1

Introduzione a R

Paolo Bosetti

Table of contents

1 Documenti Quarto (estensione .qmd)	1
1.1 Intestazione in YAML	1
1.2 Rendering: requisiti	2
1.3 Formattazione: Markdown	2
1.3.1 Questo è un titolo di livello 3	3
1.4 Elenchi puntati e numerati	3
1.5 Notebook: unire testo e codice	3
2 Richiamo basi di R	4
2.1 Vettori	4
2.2 Matrici	7
2.3 Tabelle (data.frame)	8
3 Funzioni di statistica descrittiva	10

1 Documenti Quarto (estensione .qmd)

Un documento Quarto è un file di testo che combina **testo formattato** (in Markdown) e **codice eseguibile** (in questo corso, codice R). Quando il documento viene *renderizzato*, il codice viene eseguito e i risultati (numeri, tabelle, grafici) vengono inseriti automaticamente nel documento finale (HTML, PDF, Word, ecc.).

1.1 Intestazione in YAML

Ogni documento inizia con un'intestazione delimitata da due linee ---, scritta in linguaggio YAML (un formato di testo del tipo **chiave: valore**). L'intestazione contiene i **metadati** del documento e le opzioni di rendering:

```
---
title: "Lezione 1"
subtitle: "Introduzione a R"
author: "Paolo Bosetti"
format: html
---
```

- `title`, `subtitle` e `author` definiscono titolo, sottotitolo e autore;
- `format` stabilisce il formato di output (`html`, `pdf`, `docx`, ...);
- altre opzioni utili sono ad esempio `toc: true` (indice dei contenuti), `date: today` (data di generazione) ed `embed-resources: true` (produce un unico file HTML autonomo).

Attenzione all'**indentazione**: in YAML è significativa, e le opzioni annidate vanno indentate con due spazi, ad esempio:

```
format:
  html:
    toc: true
```

Dopo i due punti lo spazio è **obbligatorio**!

1.2 Rendering: requisiti

Prima di renderizzare un documento Quarto, è necessario installare in R il *package* `rmarkdown`: dalla console di R eseguire il comando: `install.packages("rmarkdown")`.

Per creare documenti in HTML (più rapido, adatto ad una preview del contenuto), è poi sufficiente cliccare il pulsante “Preview” sulla toolbar del documento.

Per creare documenti in PDF (maggiore qualità), è necessario installare in R il package `tinytex`: dalla console di R eseguire il comando: `install.packages("tinytex")`, e successivamente eseguire `tinytex::install_tinytex()` per installare la distribuzione LaTeX completa. Questa operazione di setup va eseguita **una volta sola**. Dopodiché cliccare sul pulsante “Preview format...” e scegliere il formato “PDF (LaTeX)”

1.3 Formattazione: Markdown

Tipo di carattere: *corsivo*, **grassetto**, monospaziato (quest’ultimo per il codice).

1.3.1 Questo è un titolo di livello 3

I paragrafi vanno separati da una o più **linee vuote**.

Questo è il secondo *paragrafo*.

I commenti non compaiono nel documento finale. Per i titoli si usano uno o più # all'inizio della linea (uno per ogni livello).

1.4 Elenchi puntati e numerati

- Questo
- è un elenco
- puntato
 - con un
 - sottoelenco

Gli elenchi numerati si fanno con i numeri o le lettere:

1. Primo elemento
2. Secondo elemento: questo
è una ulteriore linea (a capo grazie al carattere \)
3. Terzo elemento
 - a. Sottoelemento
 - b. Altro sottoelemento
4. Quarto elemento

1.5 Notebook: unire testo e codice

Si può inserire un blocco di codice (*chunk*) così:

```
40 + 2
```

```
[1] 42
```

Il risultato dell'esecuzione compare subito sotto il blocco. Le scorciatoie di tastiera utili (in RStudio e Positron) sono:

- per inserire un blocco: Windows **Ctrl+Alt+I**, Mac **Cmd+Option+I**
- per eseguire la linea attiva (o la selezione): Windows **Ctrl+Invio**, Mac **Cmd+Invio**
- per eseguire l'intero blocco: Windows **Ctrl+Maiusc+Invio**, Mac **Cmd+Maiusc+Invio**

2 Richiamo basi di R

2.1 Vettori

Assegnazioni e vettori:

```
# assegnazione (si preferisce <- a =)
x <- 5
y = 3 # equivalente, ma meno usato
# vettori: la funzione c() concatena i valori
v <- c(10, 20, 30, 40, 50)
v2 <- 1:5          # sequenza di interi
v3 <- seq(0, 1, by = 0.25)
v
```

```
[1] 10 20 30 40 50
```

```
v2
```

```
[1] 1 2 3 4 5
```

```
v3
```

```
[1] 0.00 0.25 0.50 0.75 1.00
```

```
# indicizzazione di un vettore (in R gli indici partono da 1!)
v[2]          # secondo elemento
```

```
[1] 20
```

```
v[c(1, 3)]   # primo e terzo elemento
```

```
[1] 10 30
```

```
v[-1]        # tutti tranne il primo
```

```
[1] 20 30 40 50
```

```
v[2:4]      # dal secondo al quarto
```

```
[1] 20 30 40
```

```
# indicizzazione logica e filtraggio  
v > 25      # vettore di valori logici
```

```
[1] FALSE FALSE  TRUE  TRUE  TRUE
```

```
v[v > 25]   # solo gli elementi maggiori di 25
```

```
[1] 30 40 50
```

```
v[v > 15 & v < 45] # combinazione di condizioni con & (AND) e | (OR)
```

```
[1] 20 30 40
```

Operazioni vettorializzate:

```
# operazioni vettore-scalare: lo scalare è applicato a ogni elemento  
v * 2
```

```
[1] 20 40 60 80 100
```

```
v + 1
```

```
[1] 11 21 31 41 51
```

```
v^2
```

```
[1] 100 400 900 1600 2500
```

```
# operazioni vettore-vettore e riciclo degli elementi del vettore più corto  
c(1, 2, 3, 4) + c(10, 20, 30, 40) # elemento per elemento
```

```
[1] 11 22 33 44
```

```
c(1, 2, 3, 4) + c(10, 20)      # il secondo vettore viene riciclato: 10, 20, 10, 20
```

```
[1] 11 22 13 24
```

Introduciamo una funzione utile nella forma più semplice (ne vedremo il dettaglio più avanti): `sample(n)` ritorna i numeri da 1 a `n` in ordine casuale:

```
v1 <- sample(10)
v1
```

```
[1] 4 3 8 6 2 10 7 5 1 9
```

La lunghezza di un vettore si ottiene con la funzione `length()`. Per ottenere gli ultimi tre elementi di un vettore, ad esempio:

```
v <- sample(10)
v[length(v)-2:0]
```

```
[1] 2 8 3
```

Qui `2:0` è la sequenza 2, 1, 0, quindi gli indici sono `length(v)-2`, `length(v)-1` e `length(v)`. In alternativa, si può usare la funzione `tail(v, 3)`.

È anche possibile usare `length()` per **cambiare** la lunghezza di un vettore:

```
# allungare un vettore: i nuovi elementi sono NA (dato mancante)
length(v) <- 12
v
```

```
[1] 6 4 10 9 5 1 7 2 8 3 NA NA
```

```
# accorciare un vettore: gli elementi in eccesso vengono persi
length(v) <- 5
v
```

```
[1] 6 4 10 9 5
```

Note

Attenzione: le *funzioni freccia* come `length()` <- sono una particolarità di R. Non tutte le funzioni hanno l'equivalente freccia: consultare l'help in linea di una funzione per scoprire se la versione freccia (cioè con assegnazione) esiste (es: `?length`).

2.2 Matrici

Le matrici si creano con la funzione `matrix()`:

```
# per righe:  
m1 <- matrix(1:6, nrow = 2, byrow = TRUE)  
m1
```

```
      [,1] [,2] [,3]  
[1,]    1    2    3  
[2,]    4    5    6
```

```
# per colonne (comportamento predefinito)  
m2 <- matrix(1:6, nrow = 2)  
m2
```

```
      [,1] [,2] [,3]  
[1,]    1    3    5  
[2,]    2    4    6
```

L'indicizzazione avviene come per i vettori, ma passando a `[]` due indici (riga e colonna). Un indice mancante significa “intera riga/colonna”:

```
m1[2, 3]    # elemento in riga 2, colonna 3
```

```
[1] 6
```

```
m1[1, ]    # intera prima riga
```

```
[1] 1 2 3
```

```
m1[, 2]      # intera seconda colonna
```

```
[1] 2 5
```

```
m1[, c(1, 3)] # prima e terza colonna
```

```
      [,1] [,2]  
[1,]     1     3  
[2,]     4     6
```

Le operazioni aritmetiche tra matrici (+, -, *, /) avvengono elemento per elemento:

```
m1 * m2
```

```
      [,1] [,2] [,3]  
[1,]     1     6    15  
[2,]     8    20    36
```

```
m1 + 10
```

```
      [,1] [,2] [,3]  
[1,]    11    12    13  
[2,]    14    15    16
```

Per il prodotto matriciale si usa invece `%*%` (le dimensioni devono essere compatibili: colonne della prima = righe della seconda!). Per trasporre una matrice si usa `t()`:

```
m1 %*% t(m2) # (2x3) %*% (3x2) = matrice 2x2
```

```
      [,1] [,2]  
[1,]    22    28  
[2,]    49    64
```

2.3 Tabelle (data.frame)

Più utili delle matrici sono le **tabelle**, o **data.frame**: rappresentano matrici le cui colonne possono essere eterogenee (numeri, testo, valori logici, ...) e sono indicate da un **nome**.

```
# creazione di una tabella
df <- data.frame(
  A = 1:10,
  B = sample(10),
  Lettere = LETTERS[1:10]
)
df
```

	A	B	Lettere
1	1	6	A
2	2	10	B
3	3	7	C
4	4	2	D
5	5	3	E
6	6	9	F
7	7	8	G
8	8	4	H
9	9	5	I
10	10	1	J

L'indicizzazione avviene esattamente come per le matrici con `[]`, oppure estraendo un'intera colonna **per nome** come vettore:

```
df[3, ]          # terza riga
```

	A	B	Lettere
3	3	7	C

```
df[, 2]          # seconda colonna
```

```
[1] 6 10 7 2 3 9 8 4 5 1
```

```
df[3, "B"]       # elemento singolo, con la colonna indicata per nome
```

```
[1] 7
```

```
df$B             # intera colonna B, come vettore
```

```
[1] 6 10 7 2 3 9 8 4 5 1
```

```
df[["B"]] # equivalente a df$B
```

```
[1] 6 10 7 2 3 9 8 4 5 1
```

Nuove colonne possono essere create assegnandole per nome:

```
df$NuovaColonna <- df$A*2  
head(df) # mostra solo le prime righe
```

	A	B	Lettere	NuovaColonna
1	1	6	A	2
2	2	10	B	4
3	3	7	C	6
4	4	2	D	8
5	5	3	E	10
6	6	9	F	12

La colonna può essere a sua volta indicizzata, e può essere usata per filtrare le righe della tabella (indicizzazione logica):

```
df[df$B>5,]
```

	A	B	Lettere	NuovaColonna
1	1	6	A	2
2	2	10	B	4
3	3	7	C	6
6	6	9	F	12
7	7	8	G	14

3 Funzioni di statistica descrittiva

Funzioni campionarie:

```
sum(df$B^2) # somma dei quadrati
```

```
[1] 385
```

```
mean(df$B)      # media campionaria
```

```
[1] 5.5
```

```
sd(df$B)        # deviazione standard campionaria
```

```
[1] 3.02765
```

```
var(df$B)       # varianza campionaria
```

```
[1] 9.166667
```

La somma dei quadrati degli scarti dalla media si calcola ad esempio con $\sum_{i=1}^N (y_i - \bar{y})^2$

```
ss <- sum((df$B - mean(df$B))^2)
ss
```

```
[1] 82.5
```

Si noti che la varianza campionaria è proprio $ss/(N-1)$:

```
ss / (length(df$B) - 1)
```

```
[1] 9.166667
```

```
var(df$B)
```

```
[1] 9.166667
```

Attenzione ai valori NA (dati mancanti): quasi tutte le funzioni statistiche restituiscono NA se anche un solo elemento è NA, a meno di indicare esplicitamente di ignorarli con `na.rm=TRUE`:

```
v5 <- c(3, 7, 4, NA, 9)
mean(v5)                      # NA
```

```
[1] NA
```

```
mean(v5, na.rm=TRUE)      # ignora gli NA
```

```
[1] 5.75
```

```
mean(v5[!is.na(v5)])      # equivalente: filtra gli NA a mano
```

```
[1] 5.75
```

```
length(v5[!is.na(v5)])    # numero di dati validi
```

```
[1] 4
```